

Assignment 10.1

Name : Kathyayani Choudhari

Roll No : 2303A53001

Batch : 46

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

```
# Calculate average score of a student  
def calc_average(marks): total = 0  
for m in marks : total += m  
average = total / len(marks)  
return avrage # Typo here  
marks = [85, 90, 78, 92]  
print("Average Score is ", calc_average(marks))
```

Expected Output:

- Corrected and runnable Python code with explanations of the fixes.

Code:

```
Syntax and Logic Errors.py X
Syntax and Logic Errors.py > ...
1 # Act as a senior Python code reviewer.
2 # Review the following Python code and do the following:
3 # Identify all syntax errors and logical errors.
4 # Fix the code so it runs correctly.
5 # Explain each fix clearly in simple terms.
6 # Improve readability using proper indentation and PEP 8 standards.
7 # Add meaningful comments for beginners.
8 # Original code with syntax and logical errors
9 def calculate_area(radius):
10     # This function calculates the area of a circle given its radius
11     pi = 3.14 # Approximation of pi
12     area = pi * radius ** 2 # Area formula: A = πr^2
13     return area
14 # Example usage
15 radius = 5
16 area = calculate_area(radius)
17 print("The area of the circle with radius", radius, "is:", area)
18 # Fixed code with explanations
19 def calculate_area(radius):
20     """
21     This function calculates the area of a circle given its radius.
22
23     Parameters:
24     radius (float): The radius of the circle.
25
26     Returns:
27     float: The area of the circle.
28     """
29     pi = 3.14 # Approximation of pi
30     area = pi * radius ** 2 # Area formula: A = πr^2
31     return area
32 # Example usage
33 radius = 5 # Define the radius of the circle
34 area = calculate_area(radius) # Call the function to calculate the area
```

Ln 40, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Syntax and Logic Errors.py X
Syntax and Logic Errors.py > ...
19 def calculate_area(radius):
20
21     Parameters:
22     radius (float): The radius of the circle.
23
24     Returns:
25     float: The area of the circle.
26     """
27     pi = 3.14 # Approximation of pi
28     area = pi * radius ** 2 # Area formula: A = πr^2
29     return area
30
31 # Example usage
32 radius = 5 # Define the radius of the circle
33 area = calculate_area(radius) # Call the function to calculate the area
34 print("The area of the circle with radius", radius, "is:", area) #
35 # Explanation of fixes:
36 # 1. Added a docstring to the function `calculate_area` to explain what the function does, its parameters, and its return value.
37 # 2. Added comments to explain the approximation of pi and the area formula, which can help beginners understand the code.
38 # 3. Ensured that the code follows PEP 8 standards for readability, such as spacing and indentation.
39
40
41
```

Output :

```
31 This function calculates the area of a circle given its radius
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [X] ... [ ] [X]
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Syntax and Logic Errors.py"
The area of the circle with radius 5 is: 78.5
The area of the circle with radius 5 is: 78.5
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

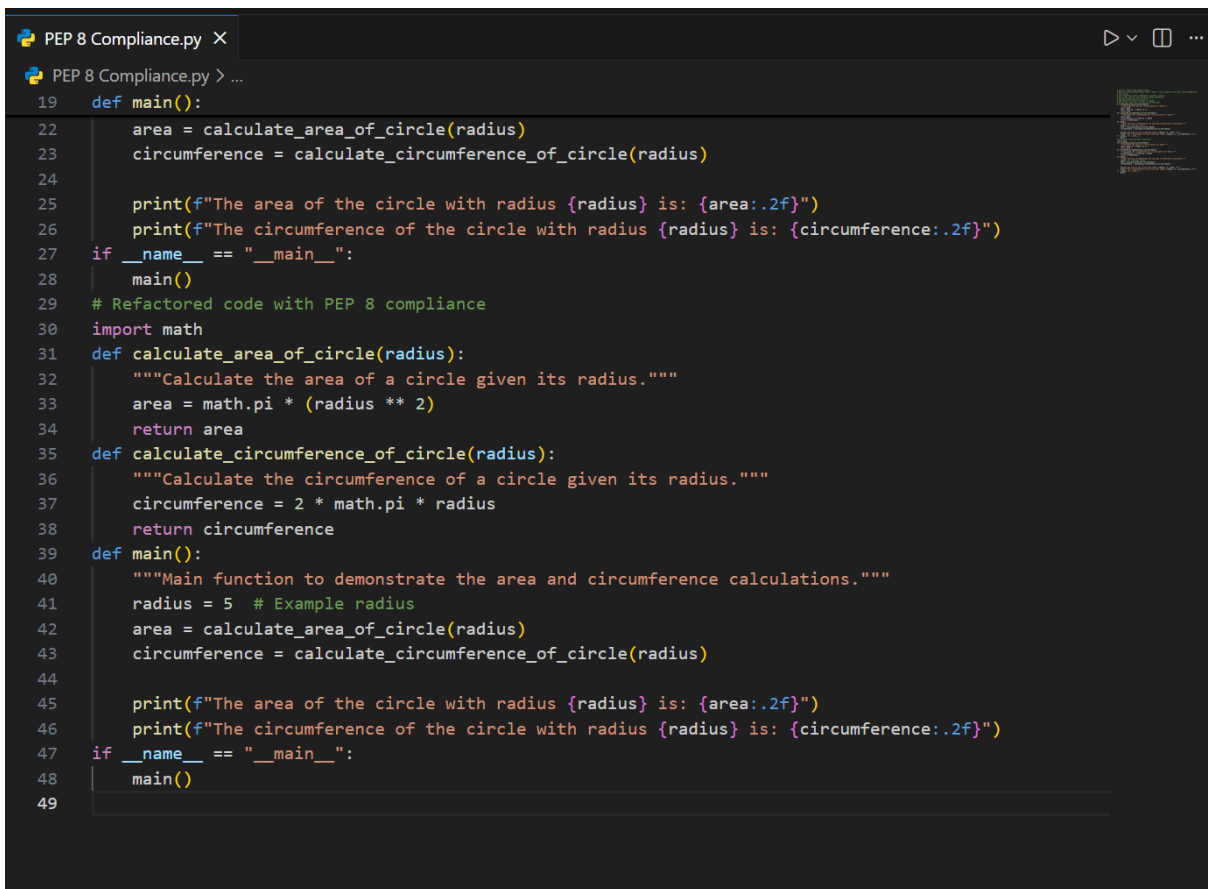
```
def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))
```

Expected Output:

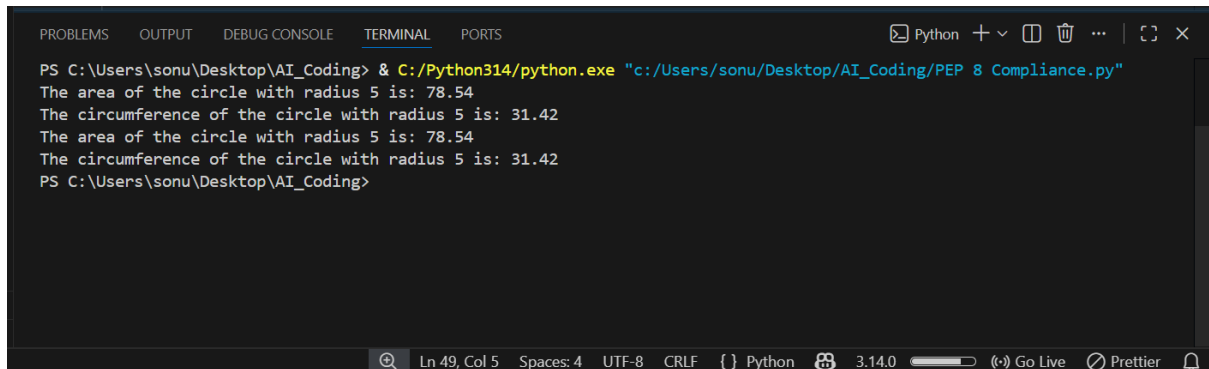
- Well-formatted PEP 8-compliant Python code.

Code:



```
PEP 8 Compliance.py X
PEP 8 Compliance.py > ...
19 def main():
20     area = calculate_area_of_circle(radius)
21     circumference = calculate_circumference_of_circle(radius)
22
23     print(f"The area of the circle with radius {radius} is: {area:.2f}")
24     print(f"The circumference of the circle with radius {radius} is: {circumference:.2f}")
25
26 if __name__ == "__main__":
27     main()
28
29 # Refactored code with PEP 8 compliance
30 import math
31
32 def calculate_area_of_circle(radius):
33     """Calculate the area of a circle given its radius."""
34     area = math.pi * (radius ** 2)
35     return area
36
37 def calculate_circumference_of_circle(radius):
38     """Calculate the circumference of a circle given its radius."""
39     circumference = 2 * math.pi * radius
40     return circumference
41
42 def main():
43     """Main function to demonstrate the area and circumference calculations."""
44     radius = 5 # Example radius
45     area = calculate_area_of_circle(radius)
46     circumference = calculate_circumference_of_circle(radius)
47
48     print(f"The area of the circle with radius {radius} is: {area:.2f}")
49     print(f"The circumference of the circle with radius {radius} is: {circumference:.2f}")
50
51 if __name__ == "__main__":
52     main()
53
54
```

Output :



```
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/PEP 8 Compliance.py"
The area of the circle with radius 5 is: 78.54
The circumference of the circle with radius 5 is: 31.42
The area of the circle with radius 5 is: 78.54
The circumference of the circle with radius 5 is: 31.42
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

```
def c(x,y):
    return x*y/100
a=200
b=15
print(c(a,b))
```

Expected Output:

- Python code with descriptive variable names, inline comments, and clear formatting.

Code:

```
Readability Enhancement.py X
Readability Enhancement.py > ...
1 # ct as a Python readability expert.
2 # Improve the readability of the following Python code without changing its logic or output.
3 # Your tasks:
4 # Replace unclear variable and function names with descriptive ones.
5 # Improve formatting and indentation.
6 # Add meaningful inline comments.
7 # Follow clean coding standards.
8 # Keep the logic exactly the same.
9 def calculate_area(radius):
10     """Calculate the area of a circle given its radius."""
11     import math
12     return math.pi * radius ** 2
13 def calculate_circumference(radius):
14     """Calculate the circumference of a circle given its radius."""
15     import math
16     return 2 * math.pi * radius
17 def main():
18     """Main function to demonstrate the area and circumference of a circle"""
19     circle_radius = 5 # Example radius
20     area = calculate_area(circle_radius)
21     circumference = calculate_circumference(circle_radius)
22
23     print(f"The area of a circle with radius {circle_radius} is: {area:.2f}")
24     print(f"The circumference of a circle with radius {circle_radius} is: {circumference:.2f}")
25 if __name__ == "__main__":
26     main()
27
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Readability Enhancement.py"

The area of a circle with radius 5 is: 78.54
The circumference of a circle with radius 5 is: 31.42
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

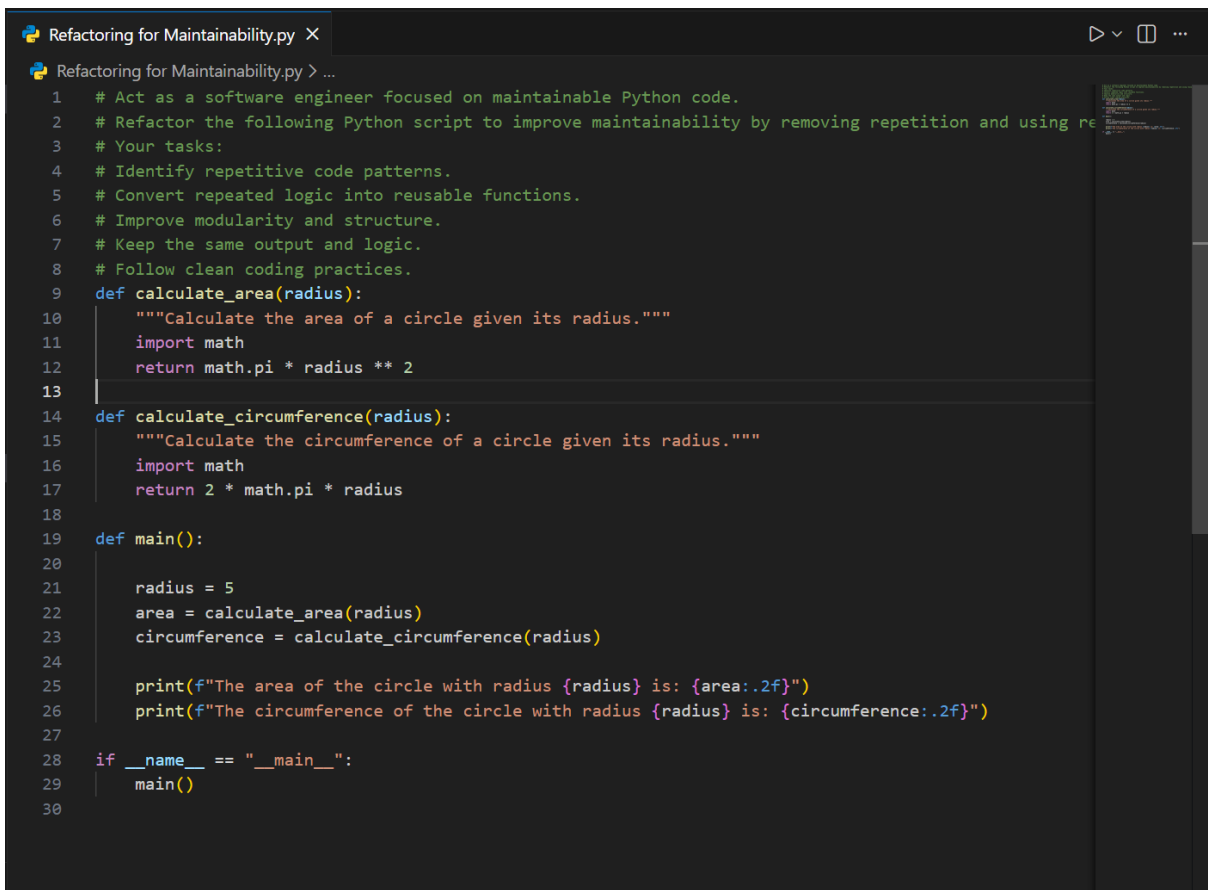
Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

Code:



```
Refactoring for Maintainability.py X
Refactoring for Maintainability.py > ...
1  # Act as a software engineer focused on maintainable Python code.
2  # Refactor the following Python script to improve maintainability by removing repetition and using reusable functions.
3  # Your tasks:
4  # Identify repetitive code patterns.
5  # Convert repeated logic into reusable functions.
6  # Improve modularity and structure.
7  # Keep the same output and logic.
8  # Follow clean coding practices.
9  def calculate_area(radius):
10     """Calculate the area of a circle given its radius."""
11     import math
12     return math.pi * radius ** 2
13
14  def calculate_circumference(radius):
15     """Calculate the circumference of a circle given its radius."""
16     import math
17     return 2 * math.pi * radius
18
19  def main():
20
21     radius = 5
22     area = calculate_area(radius)
23     circumference = calculate_circumference(radius)
24
25     print(f"The area of the circle with radius {radius} is: {area:.2f}")
26     print(f"The circumference of the circle with radius {radius} is: {circumference:.2f}")
27
28  if __name__ == "__main__":
29     main()
30
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [ ] ... [ ] [ ] X
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Refactoring for Maintainability.py"
The area of the circle with radius 5 is: 78.54
The circumference of the circle with radius 5 is: 31.42
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

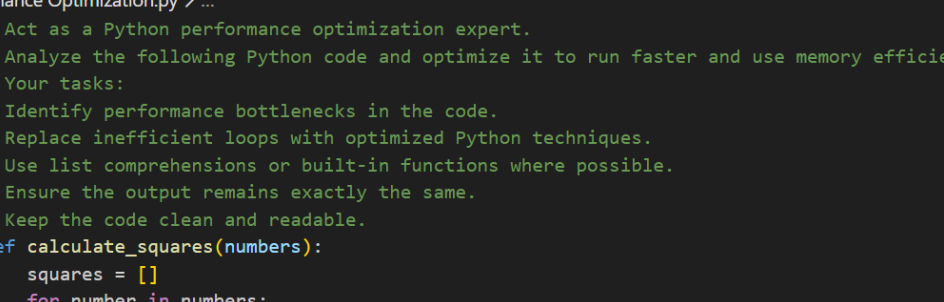
Sample Input Code:

```
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
    squares.append(n**2)
print(len(squares))
```

Expected Output:

- Optimized code using list comprehensions or vectorized operations.

Code:



The screenshot shows a code editor with a dark theme. The title bar at the top reads "Performance Optimization.py X". The editor content is a Python script with the following code:

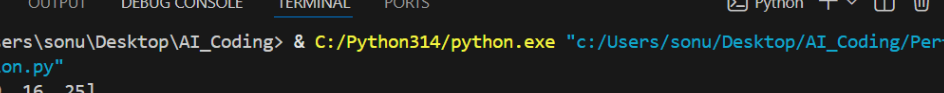
```

1  # Act as a Python performance optimization expert.
2  # Analyze the following Python code and optimize it to run faster and use memory efficiently.
3  # Your tasks:
4  # Identify performance bottlenecks in the code.
5  # Replace inefficient loops with optimized Python techniques.
6  # Use list comprehensions or built-in functions where possible.
7  # Ensure the output remains exactly the same.
8  # Keep the code clean and readable.
9  def calculate_squares(numbers):
10     squares = []
11     for number in numbers:
12         squares.append(number ** 2)
13     return squares
14  def calculate_squares(numbers):
15     return [number ** 2 for number in numbers]
16  # Example usage
17  numbers = [1, 2, 3, 4, 5]
18  squares = calculate_squares(numbers)
19  print(squares)
20  # Output: [1, 4, 9, 16, 25]
21  |
22

```

On the right side of the editor, there is a vertical sidebar with a dark background and a light blue header. It contains a search icon and some text, but it is partially obscured and difficult to read.

Output :



The screenshot shows a VS Code terminal window with the following content:

```
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Performance Optimization.py"
[1, 4, 9, 16, 25]
PS C:\Users\sonu\Desktop\AI_Coding>
```

The terminal window has a dark theme. The top bar shows tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (active), and PORTS. The bottom status bar shows the current cursor position (Ln 21, Col 1), indentation (Spaces: 4), encoding (UTF-8), line endings (CRLF), the active language (Python), version (3.14.0), and other tools like Go Live and Prettier.

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

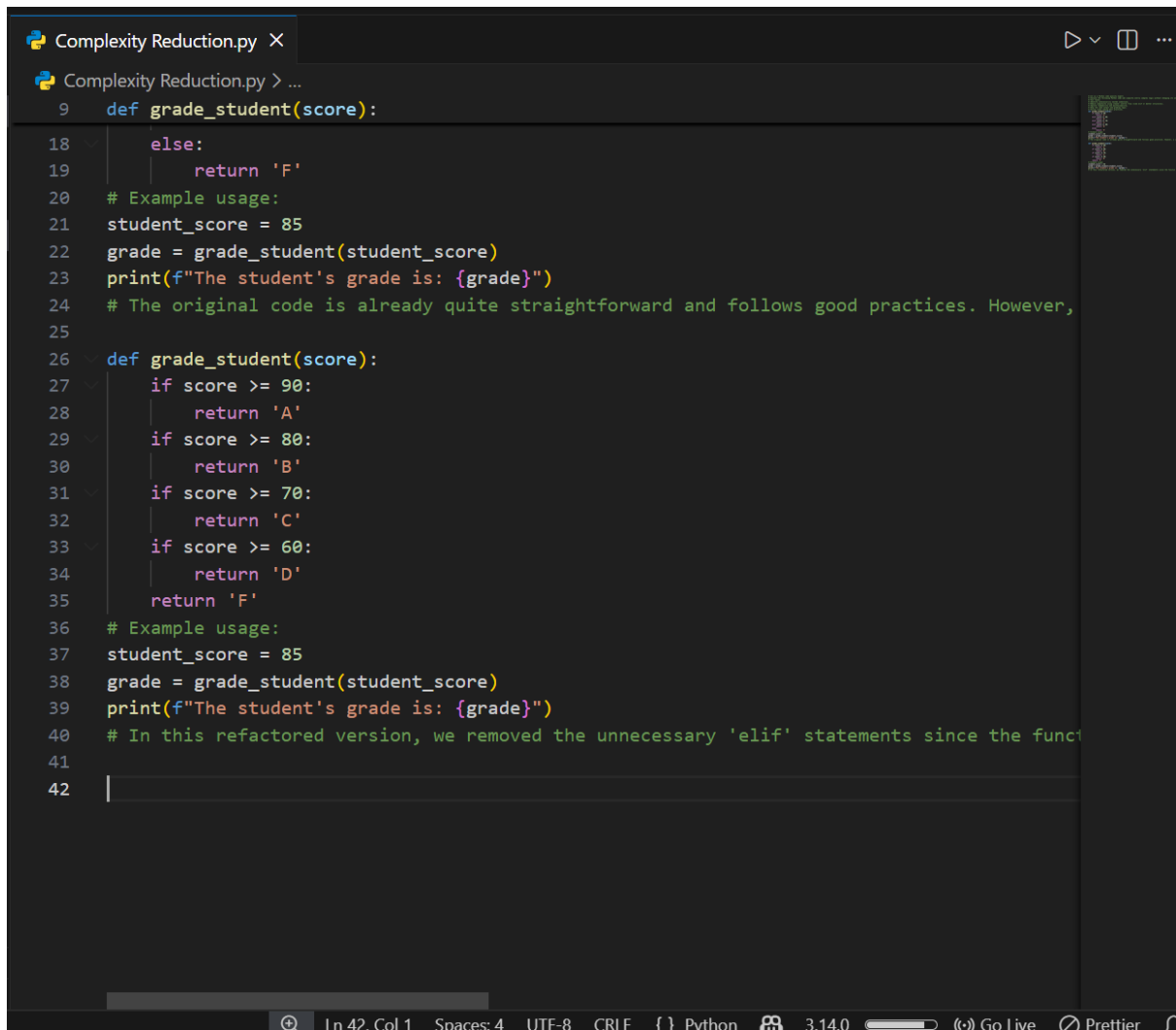
```
def grade(score):  
    if score >= 90:  
        return "A"  
    else:  
        if score >= 80:  
            return "B"  
        else:  
            if score >= 70:  
                return "C"  
            else:  
                if score >= 60:  
                    return "D"  
                else:  
                    return "F"
```

Expected Output:

- Cleaner logic using elif or dictionary mapping.

Code:

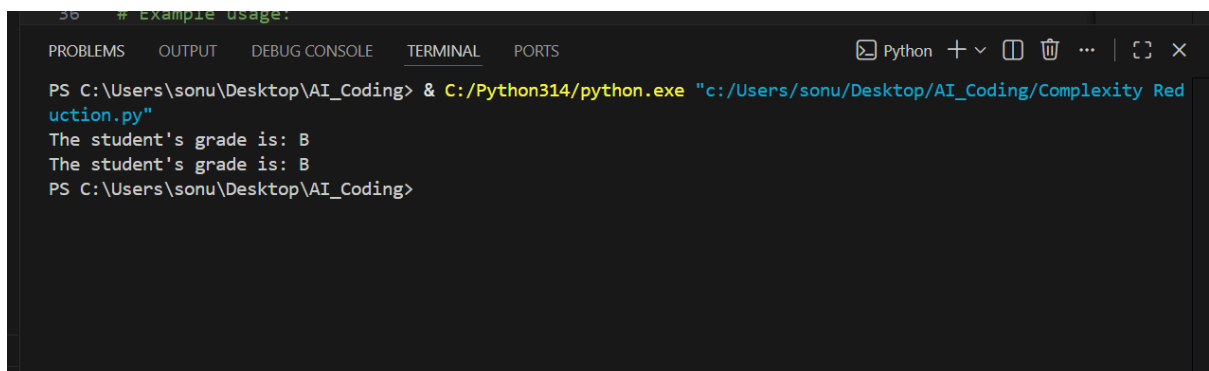
```
Complexity Reduction.py X
Complexity Reduction.py > ...
1  # Act as a Python code quality expert.
2  # Analyze the following Python code and simplify overly complex logic without changing its
3  # Your tasks:
4  # Identify unnecessarily nested conditions.
5  # Reduce complexity using cleaner control flow (like elif or better structures).
6  # Improve readability and maintainability.
7  # Keep the same output and grading logic.
8  # Follow clean coding best practices.
9  def grade_student(score):
10     if score >= 90:
11         return 'A'
12     elif score >= 80:
13         return 'B'
14     elif score >= 70:
15         return 'C'
16     elif score >= 60:
17         return 'D'
18     else:
19         return 'F'
20 # Example usage:
21 student_score = 85
22 grade = grade_student(student_score)
23 print(f"The student's grade is: {grade}")
24 # The original code is already quite straightforward and follows good practices. However,
25
26 def grade_student(score):
27     if score >= 90:
28         return 'A'
29     if score >= 80:
30         return 'B'
31     if score >= 70:
32         return 'C'
33     if score >= 60:
34         return 'D'
```



The screenshot shows a code editor with a file named 'Complexity Reduction.py'. The code defines a function 'grade_student(score)' that returns a grade based on a score. The score is set to 85, and the function is called, resulting in the output 'The student's grade is: B'. The code is commented with a note about the original code being straightforward and following good practices. The editor's status bar at the bottom shows 'Ln 42, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', '{ } Python', '3.14.0', 'Go Live', and 'Prettier'.

```
9 def grade_student(score):
18     else:
19         return 'F'
20 # Example usage:
21 student_score = 85
22 grade = grade_student(student_score)
23 print(f"The student's grade is: {grade}")
24 # The original code is already quite straightforward and follows good practices. However,
25
26 def grade_student(score):
27     if score >= 90:
28         return 'A'
29     if score >= 80:
30         return 'B'
31     if score >= 70:
32         return 'C'
33     if score >= 60:
34         return 'D'
35     return 'F'
36 # Example usage:
37 student_score = 85
38 grade = grade_student(student_score)
39 print(f"The student's grade is: {grade}")
40 # In this refactored version, we removed the unnecessary 'elif' statements since the func
41
42 |
```

Output:



The screenshot shows a terminal window with the command 'C:\Python314\python.exe "c:/Users/sonu/Desktop/AI_Coding/Complexity Red uction.py"' executed. The output is 'The student's grade is: B'.

```
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Complexity Red uction.py"
The student's grade is: B
The student's grade is: B
PS C:\Users\sonu\Desktop\AI_Coding>
```